Travaux Pratiques

Framework PHP – Laravel

Licence 3 SIO – RAZAFINDRAIBE Fabrice

TP guidés et progressifs
Windows et Linux

Fil conducteur :

Développer progressivement une application de catalogue de produits avec Laravel.

ESMIA – 2026

Table des matières

1 Organisation générale des travaux pratiques **3**
1.1 Progression proposée . . . 3
1.2 Règle de travail . . . 3

2 TP 0 – Installation et préparation de Laravel **4**
2.1 Logiciels nécessaires . . . 4
2.2 Étape 1 – Vérifier PHP . . . 4
2.3 Étape 2 – Vérifier Composer . . . 5
2.4 Étape 3 – Vérifier Node.js et NPM . . . 5
2.5 Installation sous Windows . . . 5
2.6 Installation sous Linux . . . 6
2.7 Étape 4 – Installer Laravel Installer . . . 6
2.8 Étape 5 – Créer le projet . . . 6
2.9 Étape 6 – Dépendances frontend . . . 7
2.10 Étape 7 – Observer le projet . . . 7

3 TP 1 – Première application Laravel et architecture MVC **8**
3.1 Situation . . . 8
3.2 Étape 1 – Examiner les routes existantes . . . 8
3.3 Étape 2 – Créer un contrôleur . . . 8
3.4 Étape 3 – Créer la vue . . . 9
3.5 Étape 4 – Créer la route . . . 9
3.6 Étape 5 – Tester . . . 9
3.7 Étape 6 – Modifier une donnée . . . 10
3.8 Étape 7 – Ajouter une page About . . . 10

4 TP 2 – HTTP, routage et cycle requête-réponse **11**
4.1 Fil conducteur . . . 11
4.2 Étape 1 – Créer le contrôleur . . . 11
4.3 Étape 2 – Créer les routes de base . . . 11
4.4 Étape 3 – Query string . . . 12
4.5 Étape 4 – Inspecter la requête . . . 13
4.6 Étape 5 – GET et POST sur la même URI . . . 13
4.7 Étape 6 – Générer une URL avec une route nommée . . . 14
4.8 Étape 7 – Groupe de routes Admin . . . 14
4.9 Travail d'analyse . . . 14

5 TP 3 – Contrôleurs, vues et Blade **16**
5.1 Étape 1 – Données temporaires . . . 16
5.2 Étape 2 – Créer le layout . . . 17
5.3 Étape 3 – Créer la liste des produits . . . 18
5.4 Étape 4 – Vue détail . . . 18
5.5 Étape 5 – Créer un composant Product Card . . . 19
5.6 Étape 6 – Composant Alert . . . 20
5.7 Étape 7 – Tester une liste vide . . . 20
5.8 Étape 8 – Échappement HTML . . . 20

6 TP 4 – Base de données, migrations et Eloquent **22**
6.1 Choix de la base . . . 22
6.2 Étape 1 – Configurer SQLite . . . 22
6.3 Alternative MySQL/MariaDB . . . 22
6.4 Étape 2 – Créer modèle et migration . . . 23

6.5 Étape 3 – Définir le schéma . . . 23
6.6 Étape 4 – Exécuter les migrations . . . 23
6.7 Étape 5 – Utiliser Tinker . . . 24
6.8 Étape 6 – Lire les données . . . 24
6.9 Étape 7 – Requêtes à réaliser . . . 24
6.10 Étape 8 – Utiliser Eloquent dans le contrôleur . . . 25
6.11 Étape 9 – Adapter Blade à Eloquent . . . 25
6.12 Étape 10 – Tester la réponse 404 . . . 26
6.13 Étape 11 – Faire évoluer le schéma . . . 26
6.14 Audit . . . 26

7 TP 5 – Intégration : mini-application Catalogue **27**
7.1 Cahier des charges . . . 27
7.2 Étape 1 – Route d'accueil . . . 27
7.3 Étape 2 – Liste des produits . . . 27
7.4 Étape 3 – Détail d'un produit . . . 27
7.5 Étape 4 – Recherche . . . 28
7.6 Étape 5 – Navigation . . . 28
7.7 Étape 6 – Composant Product Card . . . 28
7.8 Étape 7 – Vérifications finales . . . 28
7.9 Questions finales . . . 29

8 Proposition d'évaluation des TP **30**
8.1 Principes d'évaluation . . . 30

9 Check-list de dépannage **30**

10 Synthèse de la progression **31**

1. Organisation générale des travaux pratiques

Ces travaux pratiques accompagnent progressivement le cours sur le framework PHP Laravel. Un même projet est conservé pendant plusieurs séances :

`framework-php`

L'objectif n'est pas seulement d'apprendre des commandes, mais de comprendre :

— pourquoi chaque fichier existe ;

— quelle responsabilité il possède ;

— qui l'appelle ;

— quelles données il reçoit ;

— quelle réponse il produit.

1.1. Progression proposée

TP	Thème	Durée indicative
TP 0	Installation et vérification de l'environnement	1h30
TP 1	Première application Laravel et architecture MVC	2h00
TP 2	HTTP, routage et requête-réponse	2h30
TP 3	Contrôleurs, vues et Blade	3h00
TP 4	Base de données, migrations et Eloquent	3h00
TP 5	Intégration : mini-application Catalogue	2h30

1.2. Règle de travail

À la fin de chaque étape, l'étudiant doit :

1. exécuter l'application ;
2. vérifier le résultat dans le navigateur ou le terminal ;
3. comprendre le rôle des fichiers modifiés ;
4. répondre aux questions proposées ;
5. ne poursuivre qu'après validation de l'étape précédente.

2. TP 0 – Installation et préparation de Laravel

Objectifs

À la fin de ce TP, l'étudiant doit pouvoir :

- vérifier la présence de PHP ;
- vérifier Composer ;
- vérifier Node.js et NPM ;
- installer la commande Laravel ;
- créer une nouvelle application ;
- lancer Laravel en environnement de développement ;
- identifier les différences pratiques entre Windows et Linux.

2.1. Logiciels nécessaires

L'environnement utilisé dans ces TP nécessite au minimum :

- PHP compatible avec la version Laravel utilisée ;
- Composer ;
- Git ;
- Node.js et NPM ;
- un éditeur de code, par exemple VS Code ;
- un navigateur Web.

Dans ce cours, les exemples sont alignés sur Laravel 13.x.

2.2. Étape 1 – Vérifier PHP

Ouvrir un terminal.
Sous Windows, utiliser par exemple :

- PowerShell ;
- Windows Terminal ;
- terminal intégré de VS Code.

Sous Linux, ouvrir simplement le terminal.
Exécuter :

```
php -v
```

Point de contrôle

La commande doit afficher une version de PHP compatible avec la version de Laravel utilisée dans le cours.

Tester également :

```
php -m
```

Rechercher notamment les extensions utiles telles que :

- mbstring ;
- openssl ;
- PDO ;

— tokenizer ;
— xml ;
— ctype ;
— fileinfo.

2.3. Étape 2 – Vérifier Composer

Exécuter :

```
composer --version
```

ou :

```
composer -V
```

Questions de réflexion

Quelle est la fonction de Composer ?
Pourquoi Laravel utilise-t-il Composer plutôt qu'une série de téléchargements manuels de bibliothèques ?

2.4. Étape 3 – Vérifier Node.js et NPM

Exécuter :

```
node --version
npm --version
```

Node.js et NPM sont principalement utilisés ici pour préparer les ressources frontend.

2.5. Installation sous Windows

Si PHP, Composer et Node.js sont déjà installés sur les ordinateurs du laboratoire, passer directement aux vérifications précédentes.

Sinon, installer les logiciels avec leurs installateurs adaptés à Windows.

Après chaque installation :

1. fermer le terminal ;
2. ouvrir un nouveau terminal ;
3. vérifier la commande correspondante.

Tester :

```
php -v
composer --version
node --version
npm --version
git --version
```

Attention

Si Windows affiche `'php' n'est pas reconnu` ou `'composer' n'est pas reconnu`, le problème concerne généralement l'installation ou la variable d'environnement `PATH`.

2.6. Installation sous Linux

Sur une distribution Debian ou Ubuntu, les noms exacts des paquets peuvent varier selon la version de la distribution.

Une installation typique nécessite les éléments suivants :

```
sudo apt update

sudo apt install php php-cli php-mbstring \
php-xml php-curl php-zip php-sqlite3 \
unzip curl git composer nodejs npm
```

Puis vérifier :

```
php -v
composer --version
node --version
npm --version
git --version
```

Attention

La commande `apt` installe la version de PHP fournie par la distribution.
Avant de poursuivre, vérifier qu'elle correspond à la version requise par Laravel utilisée dans le cours.

2.7. Étape 4 – Installer Laravel Installer

Exécuter :

```
composer global require laravel/installer
```

Tester ensuite :

```
laravel --version
```

Si la commande `laravel` n'est pas trouvée, le répertoire global de Composer doit être ajouté au `PATH`.

Une solution alternative consiste à créer directement le projet avec Composer :

```
composer create-project laravel/laravel framework-php
```

2.8. Étape 5 – Créer le projet

Avec Laravel Installer :

```
laravel new framework-php
```

Entrer ensuite dans le projet :

```
cd framework-php
```

Vérifier Laravel :

```
php artisan --version
```

Puis :

```
php artisan
```

Point de contrôle

La commande `php artisan` doit afficher la liste des commandes disponibles.

2.9. Étape 6 – Dépendances frontend

Exécuter :

```
npm install
npm run build
```

Puis lancer l'environnement de développement :

```
composer run dev
```

Ouvrir ensuite dans le navigateur l'adresse locale indiquée par Laravel.
Elle est généralement proche de :

`http://localhost:8000`

2.10. Étape 7 – Observer le projet

Identifier les éléments suivants :

```
app/
bootstrap/
config/
database/
public/
resources/
routes/
storage/
tests/
vendor/

artisan
composer.json
composer.lock
.env
```

Compléter le tableau suivant :

Élément	Rôle
`app/`	
`routes/`	
`resources/views/`	
`public/`	
`database/`	
`vendor/`	
`composer.json`	
`.env`	

Livrable attendu

Une capture ou un compte rendu montrant :

- la version de PHP ;
- la version de Composer ;
- la version de Laravel ;
- l'application Laravel ouverte dans le navigateur.

3. TP 1 – Première application Laravel et architecture MVC

Objectifs

À la fin de ce TP, l'étudiant doit pouvoir :

- créer une route ;
- créer un contrôleur ;
- créer une vue Blade ;
- transmettre des données d'un contrôleur vers une vue ;
- expliquer les responsabilités de Model, View et Controller ;
- utiliser les premières commandes Artisan.

3.1. Situation

Nous allons créer la page :

`/course`

qui affichera des informations sur le cours.

3.2. Étape 1 – Examiner les routes existantes

Exécuter :

```
php artisan route:list
```

Repérer :

- la méthode HTTP ;
- l'URI ;
- le nom éventuel de la route ;
- l'action associée.

3.3. Étape 2 – Créer un contrôleur

Exécuter :

```
php artisan make:controller CourseController
```

Observer le fichier créé :

`app/Http/Controllers/CourseController.php`

Ajouter :

```
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\View\View;
6
7  class CourseController extends Controller
8  {
9      public function index(): View
10     {
11         return view('course', [
12             'title' => 'Framework PHP',
```

```
            'framework' => 'Laravel',
            'unit' => 1,
        ]);
    }
}
```

3.4. Étape 3 – Créer la vue

Créer :

`resources/views/course.blade.php`

avec :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>{{ $title }}</title>
</head>
<body>

    <h1>{{ $title }}</h1>

    <p>Framework : {{ $framework }}</p>
    <p>Unite : {{ $unit }}</p>

</body>
</html>
```

3.5. Étape 4 – Créer la route

Ouvrir :

`routes/web.php`

Ajouter :

```
use App\Http\Controllers\CourseController;

Route::get(
    '/course',
    [CourseController::class, 'index']
);
```

3.6. Étape 5 – Tester

Accéder à :

`/course`

La page doit afficher :

— Framework PHP ;

— Laravel ;

— Unité 1.

3.7. Étape 6 – Modifier une donnée

Dans le contrôleur, remplacer :

```
'unit' => 1,
```

par :

```
'unit' => 2,
```

Actualiser la page.

Point de contrôle

L'étudiant doit comprendre que la donnée est préparée dans le contrôleur et affichée par la vue.

3.8. Étape 7 – Ajouter une page About

Ajouter dans le même contrôleur :

```
public function about(): View
{
    return view('about');
}
```

Créer :

`resources/views/about.blade.php`

et ajouter la route :

```
Route::get(
    '/about',
    [CourseController::class, 'about']
);
```

Questions de réflexion

Répondre aux questions suivantes :

1. Quel fichier reçoit la définition de la route ?
2. Quel fichier coordonne le traitement ?
3. Quel fichier contient le HTML ?
4. Pourquoi ne met-on pas tout dans `routes/web.php` ?
5. Faut-il obligatoirement créer un nouveau contrôleur pour chaque URL ?
6. Que signifie la séparation des responsabilités ?

Livrable attendu

Le projet doit contenir :

- `CourseController` ;
- la vue `course.blade.php` ;
- la vue `about.blade.php` ;
- les routes `/course` et `/about`.

4. TP 2 – HTTP, routage et cycle requête-réponse

Objectifs

À la fin de ce TP, l'étudiant doit pouvoir :

- distinguer GET et POST ;
- créer plusieurs routes Laravel ;
- utiliser un paramètre de route ;
- utiliser une query string ;
- imposer une contrainte à un paramètre ;
- utiliser l'objet `Request` ;
- créer une réponse JSON ;
- utiliser des routes nommées ;
- créer un groupe de routes.

4.1. Fil conducteur

Nous allons commencer un catalogue de produits sans utiliser encore de base de données.

4.2. Étape 1 – Créer le contrôleur

Exécuter :

```
php artisan make:controller ProductController
```

Ajouter :

```
public function index()
{
    return 'Liste des produits';
}

public function show(string $id)
{
    return "Produit $id";
}
```

4.3. Étape 2 – Créer les routes de base

Ajouter dans `routes/web.php` :

```
use App\Http\Controllers\ProductController;

Route::get(
    '/products',
    [ProductController::class, 'index']
)->name('products.index');

Route::get(
    '/products/{id}',
    [ProductController::class, 'show']
)
->whereNumber('id')
->name('products.show');
```

Exécuter :

```
php artisan route:list
```

Tester :

`/products`

puis :

`/products/10`

et enfin :

`/products/test`

Questions de réflexion

Pourquoi `/products/test` ne produit-il pas le même résultat que `/products/10` ?

4.4. Étape 3 – Query string

Créer une route :

```
use Illuminate\Http\Request;

Route::get('/search', function (Request $request) {

    $query = $request->query('q', '');

    return "Recherche : $query";
});
```

Tester :

`/search?q=laravel`

puis :

`/search?q=php`

Questions de réflexion

Comparer :

`/products/42`

et :

`/search?q=laravel`

Quelle information appartient au chemin ?
Quelle information appartient à la query string ?

4.5. Étape 4 – Inspecter la requête

Créer :

```php
Route::get('/debug/request', function (Request $request) {

    return response()->json([
        'method' => $request->method(),
        'path' => $request->path(),
        'query' => $request->query(),
        'accept' => $request->header('Accept'),
    ]);
});
```

Tester dans le navigateur :

`/debug/request?test=123`

Identifier les quatre informations retournées.

4.6. Étape 5 – GET et POST sur la même URI

Créer une vue :

`resources/views/contact.blade.php`

contenant :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Contact</title>
</head>
<body>

<h1>Contact</h1>

<form method="POST" action="/contact">
    @csrf

    <label>Nom :</label>
    <input type="text" name="name">

    <button type="submit">Envoyer</button>
</form>

</body>
</html>
```

Ajouter :

```php
Route::get('/contact', function () {
    return view('contact');
});

Route::post('/contact', function (Request $request) {
    return 'Formulaire recu par POST';
});
```

Tester le formulaire.

Questions de réflexion

Pourquoi Laravel peut-il avoir deux routes `/contact` différentes ?
Quelle information permet de les distinguer ?

4.7. Étape 6 – Générer une URL avec une route nommée

Tester temporairement :

```
Route::get('/test-url', function () {

    return route(
        'products.show',
        ['id' => 5]
    );
});
```

Observer l'URL obtenue.

4.8. Étape 7 – Groupe de routes Admin

Ajouter :

```
Route::prefix('admin')
    ->name('admin.')
    ->group(function () {

        Route::get('/dashboard', function () {
            return 'Dashboard';
        })->name('dashboard');

        Route::get('/products', function () {
            return 'Admin products';
        })->name('products');
    });
```

Exécuter :

```
php artisan route:list
```

Repérer :

- `admin.dashboard` ;
- `admin.products`.

4.9. Travail d'analyse

Pour chaque besoin, proposer la méthode HTTP la plus logique :

Opération	Méthode
Afficher tous les produits	
Afficher le produit 15	
Créer un nouveau produit	
Modifier partiellement le produit 15	
Supprimer le produit 15	

Questions de réflexion

Répondre :

1. Quelle différence entre URI et nom de route ?
2. Quelle différence entre `{id}` et `?page=2` ?
3. À quoi sert `whereNumber()` ?
4. Pourquoi GET et POST peuvent-ils avoir la même URI ?
5. Quel objet représente la requête courante ?
6. Comment Laravel peut-il retourner du JSON ?
7. Quelle est l'utilité de `route:list` ?

5. TP 3 – Contrôleurs, vues et Blade

Objectifs

À la fin de ce TP, l'étudiant doit pouvoir :

- transmettre des données d'un contrôleur vers une vue ;
- utiliser les expressions Blade ;
- utiliser les conditions et boucles Blade ;
- utiliser `@forelse` ;
- construire un layout réutilisable ;
- créer un composant Blade ;
- utiliser des props et un slot ;
- comprendre l'échappement automatique.

5.1. Étape 1 – Données temporaires

Modifier `ProductController` :

```php
<?php

namespace App\Http\Controllers;

use Illuminate\View\View;

class ProductController extends Controller
{
    private array $products = [
        [
            'id' => 1,
            'name' => 'Laptop',
            'price' => 2500,
            'stock' => 10,
            'description' => 'Ordinateur portable'
        ],
        [
            'id' => 2,
            'name' => 'Clavier',
            'price' => 120,
            'stock' => 0,
            'description' => 'Clavier USB'
        ],
        [
            'id' => 3,
            'name' => 'Souris',
            'price' => 80,
            'stock' => 25,
            'description' => 'Souris optique'
        ],
    ];

    public function index(): View
    {
        return view('products.index', [
            'products' => $this->products
        ]);
    }
```

```
39
40      public function show(string $id): View
41      {
42          $product = collect($this->products)
43              ->firstWhere('id', (int) $id);
44
45          abort_if($product === null, 404);
46
47          return view('products.show', [
48              'product' => $product
49          ]);
50      }
51  }
```

5.2. Étape 2 – Créer le layout

Créer :

resources/views/components/layout.blade.php

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">

    <title>{{ $title ?? 'Catalogue' }}</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            max-width: 900px;
            margin: 30px auto;
        }

        nav a {
            margin-right: 15px;
        }

        .product {
            border: 1px solid #ccc;
            padding: 15px;
            margin: 10px 0;
        }
    </style>
</head>

<body>

<nav>
    <a href="{{ route('products.index') }}">
        Produits
    </a>
</nav>

<hr>

<main>
    {{ $slot }}
</main>
```

```
<hr>

<footer>
    Framework PHP -- Laravel
</footer>

</body>
</html>
```

5.3. Étape 3 – Créer la liste des produits

Créer :

resources/views/products/index.blade.php

```
<x-layout title="Catalogue">

    <h1>Catalogue des produits</h1>

    @forelse($products as $product)

        <div class="product">

            <h2>{{ $product['name'] }}</h2>

            <p>
                Prix : {{ $product['price'] }}
            </p>

            @if($product['stock'] > 0)
                <p>Disponible</p>
            @else
                <p>Rupture de stock</p>
            @endif

            <a href="{{ route(
                'products.show',
                ['id' => $product['id']]
            ) }}">
                Voir le produit
            </a>

        </div>

    @empty

        <p>Aucun produit disponible.</p>

    @endforelse

</x-layout>
```

5.4. Étape 4 – Vue détail

Créer :

resources/views/products/show.blade.php

```
<x-layout :title="$product['name']">

    <h1>{{ $product['name'] }}</h1>

    <p>
        Prix : {{ $product['price'] }}
    </p>

    <p>
        Stock : {{ $product['stock'] }}
    </p>

    <p>
        {{ $product['description'] }}
    </p>

    <p>
        <a href="{{ route('products.index') }}">
            Retour au catalogue
        </a>
    </p>

</x-layout>
```

5.5. Étape 5 – Créer un composant Product Card

Créer :

resources/views/components/product-card.blade.php

```
@props([
    'name',
    'price',
    'stock'
])

<div {{ $attributes->class(['product']) }}>

    <h2>{{ $name }}</h2>

    <p>Prix : {{ $price }}</p>

    @if($stock > 0)
        <p>Disponible</p>
    @else
        <p>Rupture</p>
    @endif

    {{ $slot }}

</div>
```

Modifier ensuite la page index pour utiliser :

```
<x-product-card
    :name="$product['name']"
    :price="$product['price']"
    :stock="$product['stock']"
>
```

```
    <a href="{{ route(
        'products.show',
        ['id' => $product['id']]
    ) }}">
        Voir
    </a>

</x-product-card>
```

5.6. Étape 6 – Composant Alert

Créer :

resources/views/components/alert.blade.php

```
@props([
    'type' => 'success'
])

<div {{ $attributes }}>
    <strong>{{ $type }}</strong>
    {{ $slot }}
</div>
```

Tester :

```
<x-alert type="warning" class="message">
    Attention : stock faible.
</x-alert>
```

5.7. Étape 7 – Tester une liste vide

Dans le contrôleur, remplacer temporairement :

```
'products' => $this->products
```

par :

```
'products' => []
```

La page doit afficher :

Aucun produit disponible.

Remettre ensuite les produits.

5.8. Étape 8 – Échappement HTML

Ajouter temporairement :

```
[
    'id' => 4,
    'name' => '<script>alert("test")</script>',
    'price' => 10,
    'stock' => 1,
    'description' => 'Produit de test'
]
```

Afficher normalement :

```
{{ $product['name'] }}
```

Observer le résultat.

Attention

Blade échappe normalement les données affichées avec `{{ ... }}`.
Ne pas transformer l'exercice en vulnérabilité en utilisant inutilement une sortie HTML non échappée.

Questions de réflexion

Répondre :

1. Pourquoi séparer contrôleur et vue ?
2. Que contient `resources/views` ?
3. Que fait le helper `view()` ?
4. Que signifie `products.index` ?
5. Pourquoi utiliser `@forelse` ?
6. À quoi sert `$slot` ?
7. Qu'est-ce qu'une prop ?
8. Pourquoi éviter les requêtes SQL dans Blade ?
9. Pourquoi l'échappement HTML est-il important ?

6. TP 4 – Base de données, migrations et Eloquent

Objectifs

À la fin de ce TP, l'étudiant doit pouvoir :

- configurer une base de données Laravel ;
- créer une migration ;
- exécuter et annuler une migration ;
- créer un modèle Eloquent ;
- insérer des données avec Tinker ;
- lire et filtrer des données avec Eloquent ;
- intégrer Eloquent dans un contrôleur ;
- utiliser `findOrFail()`.

6.1. Choix de la base

Pour le TP, SQLite est pratique car la base est contenue dans un fichier.
MySQL ou MariaDB peuvent également être utilisés dans le laboratoire.

6.2. Étape 1 – Configurer SQLite

Dans `.env` :

```
DB_CONNECTION=sqlite
```

Créer ensuite le fichier de base.
Sous Linux :

```
touch database/database.sqlite
```

Sous Windows PowerShell :

```
New-Item database\database.sqlite -ItemType File -Force
```

Attention

Le fichier `.env` peut contenir des informations sensibles.
Les mots de passe de production ne doivent pas être publiés dans un dépôt Git public.

6.3. Alternative MySQL/MariaDB

Une configuration typique est :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=framework_php
DB_USERNAME=root
DB_PASSWORD=
```

Les valeurs exactes dépendent du laboratoire.

6.4. Étape 2 – Créer modèle et migration

Exécuter :

```
php artisan make:model Product -m
```

Vérifier la création de :

— `app/Models/Product.php` ;
— une migration dans `database/migrations/`.

6.5. Étape 3 – Définir le schéma

Dans la migration :

```
Schema::create('products', function (Blueprint $table) {

    $table->id();

    $table->string('name');

    $table->string('sku')->unique();

    $table->decimal('price', 10, 2);

    $table->unsignedInteger('stock')
          ->default(0);

    $table->text('description')
          ->nullable();

    $table->timestamps();
});
```

Identifier les colonnes :

— id ;
— name ;
— sku ;
— price ;
— stock ;
— description ;
— created_at ;
— updated_at.

6.6. Étape 4 – Exécuter les migrations

Exécuter :

```
php artisan migrate
```

Puis :

```
php artisan migrate:status
```

Point de contrôle

La migration créant la table `products` doit apparaître comme exécutée.

6.7. Étape 5 – Utiliser Tinker

Exécuter :

```
php artisan tinker
```

Créer un produit :

```
$product = new App\Models\Product();

$product->name = 'Laptop';
$product->sku = 'PC-001';
$product->price = 2500;
$product->stock = 10;
$product->description = 'Ordinateur portable';

$product->save();
```

Créer au moins quatre produits supplémentaires.
Exemple :

```
$p = new App\Models\Product();
$p->name = 'Clavier';
$p->sku = 'CL-001';
$p->price = 120;
$p->stock = 0;
$p->save();
```

6.8. Étape 6 – Lire les données

Dans Tinker, exécuter :

```
App\Models\Product::all();
```

Puis :

```
App\Models\Product::where(
    'stock',
    '>',
    0
)->get();
```

Tester également :

```
App\Models\Product::where(
    'price',
    '>',
    1000
)->get();
```

et :

```
App\Models\Product::orderBy(
    'price'
)->get();
```

6.9. Étape 7 – Requêtes à réaliser

Écrire la requête Eloquent permettant d'obtenir :

1. tous les produits ;
2. les produits disponibles ;

3. les produits de plus de 1000 ;
4. les produits triés par prix ;
5. le produit ayant le SKU `PC-001` ;
6. le nombre total de produits ;
7. le prix moyen.

Pour chaque requête, écrire également l'équivalent SQL approximatif.

6.10. Étape 8 – Utiliser Eloquent dans le contrôleur

Dans `ProductController` :

```
use App\Models\Product;
```

Modifier `index()` :

```
public function index(): View
{
    $products = Product::orderBy('name')->get();

    return view('products.index', [
        'products' => $products
    ]);
}
```

Modifier `show()` :

```
public function show(string $id): View
{
    $product = Product::findOrFail($id);

    return view('products.show', [
        'product' => $product
    ]);
}
```

6.11. Étape 9 – Adapter Blade à Eloquent

Les produits ne sont plus des tableaux PHP.
Remplacer par exemple :

```
{{ $product['name'] }}
```

par :

```
{{ $product->name }}
```

Même principe pour :

- `$product->price` ;
- `$product->stock` ;
- `$product->description` ;
- `$product->id`.

6.12. Étape 10 – Tester la réponse 404

Tester une ressource existante :

`/products/1`

Puis une ressource inexistante :

`/products/999999`

Questions de réflexion

Pourquoi `findOrFail()` est-il particulièrement adapté à la page de détail d'une ressource ?

6.13. Étape 11 – Faire évoluer le schéma

Créer une nouvelle migration :

```
php artisan make:migration add_active_to_products_table
```

Dans `up()` :

```
Schema::table('products', function (Blueprint $table) {
    $table->boolean('active')->default(true);
});
```

Dans `down()` :

```
Schema::table('products', function (Blueprint $table) {
    $table->dropColumn('active');
});
```

Exécuter :

```
php artisan migrate
```

Puis :

```
php artisan migrate:rollback
```

Vérifier la disparition de la colonne.
Enfin :

```
php artisan migrate
```

6.14. Audit

Pour chaque proposition, répondre **acceptable** ou **mauvaise pratique**, puis justifier.

1. Modifier directement une table sans créer de migration.
2. Mettre `DB_PASSWORD` dans un dépôt Git public.
3. Utiliser `findOrFail()` pour afficher le détail d'un produit.
4. Construire une requête SQL par concaténation d'une entrée utilisateur.
5. Utiliser une contrainte `UNIQUE` pour une référence produit réellement unique.
6. Exécuter `migrate:fresh` sur une base de production sans précaution.

7. TP 5 – Intégration : mini-application Catalogue

Objectifs

Ce TP final réunit les notions étudiées dans les TP précédents.
L'étudiant doit être capable de construire une fonctionnalité sans recevoir chaque ligne de code.

7.1. Cahier des charges

L'application doit proposer :

1. une page d'accueil ;
2. une page listant les produits ;
3. une page de détail ;
4. une recherche ;
5. une base de données persistante ;
6. un layout commun ;
7. un composant d'affichage des produits ;
8. des routes nommées ;
9. une gestion correcte d'une ressource inexistante.

7.2. Étape 1 – Route d'accueil

Créer :

```
GET /
```

La page doit contenir :

— le titre de l'application ;
— un court texte de présentation ;
— un lien vers le catalogue.

7.3. Étape 2 – Liste des produits

La route :

```
GET /products
```

doit être nommée :

```
products.index
```

Les produits sont récupérés avec Eloquent.
Ils sont triés par nom.

7.4. Étape 3 – Détail d'un produit

La route :

```
GET /products/{id}
```

doit :

— utiliser une contrainte numérique ;
— être nommée `products.show` ;
— utiliser `findOrFail()`.

7.5. Étape 4 – Recherche

Créer :

```
GET /search?q=...
```

Le contrôleur récupère :

```
$request->query('q', '')
```

puis recherche les produits dont le nom correspond au texte demandé.
Résultat attendu :

— afficher les produits correspondants ;

— afficher un message si aucun produit n'est trouvé.

7.6. Étape 5 – Navigation

Le layout doit proposer au minimum :

— Accueil ;

— Produits ;

— Recherche.

Les liens internes doivent utiliser autant que possible les noms de routes.

7.7. Étape 6 – Composant Product Card

Le composant produit doit afficher :

— nom ;

— prix ;

— état du stock ;

— lien vers la fiche détaillée.

La présentation du produit ne doit pas être dupliquée inutilement.

7.8. Étape 7 – Vérifications finales

Exécuter :

```
php artisan route:list
```

Puis tester :

1. `/` ;
2. `/products` ;
3. un produit existant ;
4. un identifiant inexistant ;
5. un identifiant non numérique ;
6. une recherche avec résultat ;
7. une recherche sans résultat.

7.9. Questions finales

Questions de réflexion

Chaque étudiant répond individuellement :

1. Quel est le rôle du routeur ?
2. Quel est le rôle d'un contrôleur ?
3. Quel est le rôle d'une vue ?
4. Quel est le rôle d'un modèle Eloquent ?
5. Pourquoi les vues ne devraient-elles pas exécuter directement les requêtes SQL ?
6. Pourquoi utiliser des routes nommées ?
7. Quel est l'intérêt des migrations ?
8. Quelle différence entre route parameter et query string ?
9. Quelle différence entre `find()` et `findOrFail()` ?
10. Comment circule une requête depuis le navigateur jusqu'à la réponse HTML ?

8. Proposition d'évaluation des TP

Critère	Points
Installation et environnement fonctionnel	2
Routes et compréhension HTTP	3
Organisation contrôleur / vue	3
Utilisation correcte de Blade	3
Migration et modèle Eloquent	3
Fonctionnement global de l'application	3
Qualité et organisation du code	2
Réponses aux questions d'analyse	1
Total	**20**

8.1. Principes d'évaluation

La note ne doit pas porter uniquement sur le fait que "l'application fonctionne".
L'étudiant doit également être capable d'expliquer :

— ce qu'il a créé ;

— pourquoi le fichier existe ;

— comment Laravel l'utilise ;

— la responsabilité de chaque composant.

9. Check-list de dépannage

Lorsqu'une application ne fonctionne pas, procéder dans cet ordre.

1. Vérifier l'environnement

```
php -v
composer --version
php artisan --version
```

2. Vérifier les routes

```
php artisan route:list
```

3. Vérifier le terminal Laravel

Lire le message d'erreur complet.

4. Vérifier les noms

Contrôler :

— nom du contrôleur ;

— namespace ;

— nom de méthode ;

— nom de vue ;

— nom de route ;

— nom de variable.

5. Vérifier la base

```
php artisan migrate:status
```

6. Vérifier les caches si nécessaire

Après certaines modifications de configuration :

```
php artisan optimize:clear
```

10. Synthèse de la progression

À travers les différents TP, l'application a évolué de la manière suivante :

PHP et Composer
↓
Projet Laravel
↓
Routes
↓
Contrôleurs
↓
Vues Blade
↓
Composants Blade
↓
Migration
↓
Modèle Eloquent
↓
Base de données persistante

L'objectif final est que l'étudiant ne voie pas Laravel comme une simple collection de commandes, mais comme une architecture organisée dans laquelle chaque composant possède une responsabilité identifiable.

Question réflexe :

Pourquoi cette classe existe-t-elle ?
Quelle responsabilité possède-t-elle ?
Qui l'appelle ?
Quelles données reçoit-elle ?
Que retourne-t-elle ?